

Info Extractor Project Report

- **Project: Info Extractor** <Benedict Kam, s5>
-

▼ Table of Contents

- Introduction
 - Methodologies
 - Discussion
 - Conclusion
 - Recommendations
 - References
-

Introduction

- In the realm of cybersecurity, situational awareness is paramount. A comprehensive understanding of a system's configuration, resource utilization, and active services is crucial for identifying vulnerabilities, detecting anomalies, and responding effectively to security incidents. This report explores the development and implementation of an "Info Extractor" bash script designed to automate the collection of critical system information within a Linux environment. The primary aim of this report is to demonstrate the creation and utility of a bash script that streamlines the process of gathering essential system data. By automating this data collection, security professionals can gain valuable insights into the system's security posture and identify potential areas of concern
-

Methodologies

1. Retrieve Public IP Address

- `curl https://api.ipify.org` : This command utilizes the `curl` utility to make an HTTP request to the `ipify.org` API. This service provides a simple way to retrieve the machine's public IP address as seen from the internet. The output is then displayed on the terminal.

2. Retrieve Internal IP Address

- `ifconfig | grep inet | awk '{print $2}' | head -1` : This command pipeline uses `ifconfig` to display network interface configuration. The output is then piped to `grep inet` to filter lines containing "inet," which indicates IPv4 addresses. `awk '{print $2}'` extracts the second field (the IP address) from these lines. Finally, `head -1` selects only the first IP address, which is typically the primary internal IP address of the machine.

3. Retrieve and Censor MAC Address

- `mac=$(ifconfig | awk '/ether/ {print $2}')` : This command first retrieves the MAC address using `ifconfig` and `awk`, similar to how the internal IP address is obtained. The MAC address is stored in the `mac` variable.
- `censored_mac=$(echo $mac | sed 's/^\(..:..:..\):..:..$/\1XX:XX:XX/')` : This command uses `sed` to censor the MAC address stored in the `mac` variable. The regular expression matches the MAC address format and replaces the last three octets with "XX" to protect privacy.

4. Display Top 5 Processes by CPU Usage

- `ps -eo pid,comm,%cpu --sort=-%cpu | head -n 6` : This command uses `ps` to list processes with their process ID (PID), command name (`comm`), and CPU usage (`%cpu`). The `--sort=-%cpu` option sorts the processes in descending order of CPU usage. `head -n 6` displays the top 6 lines, which include the header and the top 5 CPU-consuming processes.

5. Display Memory Usage

- `free -h | awk '/^Mem:/ {print "Used: "$3, "\nFree: "$4}' | sed 's/Gi/GB/g; s/Mi/MB/g'` : This command pipeline uses `free -h` to display memory usage in a human-readable format (with units like GB and MB). `awk` is used to extract the used and free memory values, and `sed` replaces "Gi" with "GB" and "Mi" with "MB" for consistency.

6. Display Active Services and Status

- `systemctl list-units --type=service --state=running --no-pager` : This command uses `systemctl` to list all running services. The `-type=service` option filters for services, `-state=running` selects only those that are currently running, and `-no-pager` prevents the output from being displayed through a pager, ensuring it's printed directly to the terminal.

7. Display Top 10 Largest Files

- `find /home -type f -exec du -h {} + 2>/dev/null | sort -rh | head -n 10` : This command pipeline uses `find` to locate all regular files (`type f`) within the `/home` directory. `exec du -h {} +` executes the `du -h` command on those files to display their sizes in human-readable format. `2>/dev/null` redirects any error messages to suppress them. The output is then piped to `sort -rh` to sort the files by size in descending order, and `head -n 10` displays the top 10 largest files.

Discussion

- **Network Configuration:**

- **Public IP Address:** Knowing the public IP address allows for the identification of the system's exposure to the internet. This information can be used to assess the risk of external attacks and the need for firewall rules or intrusion detection systems to protect against unauthorized access.
- **Internal IP Address:** The internal IP address helps understand the system's position within the local network. This is crucial for network segmentation and access control, ensuring that sensitive systems are isolated from less secure parts of the network.
- **MAC Address (Censored):** While the complete MAC address is sensitive information, the censored version provides some level of identification without compromising privacy. This can be useful for network monitoring and intrusion detection, as unusual MAC addresses or changes in MAC addresses can indicate unauthorized devices or network spoofing attempts.

- **Resource Utilization:**

- **Top CPU Consuming Processes:** Monitoring CPU usage helps identify processes that are consuming excessive resources. This can reveal potential malware activity, denial-of-service attacks, or resource exhaustion issues that could impact system stability and availability.
- **Memory Usage:** Analyzing memory usage is crucial for detecting memory leaks or excessive memory consumption, which can lead to system instability, crashes, or denial-of-service vulnerabilities.

- **System Activity:**

- **Active Services and Status:** Identifying active services allows for the assessment of potential vulnerabilities associated with those services. Unnecessary or outdated services can pose security risks, and monitoring their status helps detect unauthorized changes or suspicious activity.
- **Largest Files in `/home`:** Examining the largest files in the user's home directory can reveal potential data hoarding, unauthorized file sharing, or the presence of large malware files. It also helps identify sensitive data that may require additional protection.

By automating the collection of this information, the "Info Extractor" script enables security professionals to proactively monitor system health, detect anomalies, and identify potential vulnerabilities that could be exploited by attackers. This information can then be used to implement appropriate security measures and incident response plans to mitigate risks and ensure the confidentiality, integrity, and availability of critical systems and data.

Conclusion

- From a cybersecurity standpoint, the "Info Extractor" script demonstrates the value of automated system analysis in proactively identifying potential vulnerabilities and maintaining a robust security posture. By automatically gathering key system information, security professionals can gain valuable insights into network configuration, resource utilization, and system activity.
 - The script's ability to extract public and internal IP addresses, along with a censored MAC address, provides crucial information for assessing external and internal network security risks. Monitoring CPU and memory usage helps detect anomalies that could indicate malware activity or resource exhaustion attacks. Furthermore, the identification of active services and analysis of file sizes in the user's home directory allows for the detection of unauthorized changes, suspicious activity, and potential data breaches.
 - However, it is essential to acknowledge that this script provides a snapshot of the system's state at a particular moment. Continuous monitoring and analysis are crucial for detecting dynamic threats and maintaining ongoing security awareness.
 - The findings of this analysis underscore the need for a multi-layered security approach that combines automated tools like the "Info Extractor" script with proactive security measures, regular vulnerability assessments, and user education. By leveraging these tools and strategies, organizations can strengthen their defenses against cyber threats and protect critical assets.
-

Recommendations

Based on the analysis conducted and the information extracted by the "Info Extractor" script, the following recommendations are proposed to enhance system security and mitigate potential vulnerabilities:

- **Network Hardening:**
 - **Firewall Implementation:** Configure a firewall to restrict incoming and outgoing network traffic based on the identified public IP address. Implement rules to allow only necessary connections and block unauthorized access attempts.
 - **Network Segmentation:** Utilize the internal IP address information to segment the network and isolate sensitive systems from less secure areas. This limits the impact of potential breaches and prevents lateral movement by attackers.
 - **MAC Address Filtering:** Consider implementing MAC address filtering on network devices to restrict access to known and authorized devices. While MAC addresses can be spoofed, this adds an extra layer of security.
- **Resource Monitoring and Management:**
 - **Performance Monitoring:** Implement system monitoring tools to continuously track CPU and memory usage. Set alerts for unusual spikes or sustained high usage that could indicate malicious activity or resource exhaustion attacks.
 - **Process Analysis:** Regularly review the list of top CPU-consuming processes to identify and investigate any unfamiliar or suspicious processes. This can help detect malware or unauthorized applications running on the system.
- **System Hardening and Maintenance:**
 - **Service Auditing:** Regularly audit active services and disable any unnecessary or outdated services to reduce the attack surface. Keep remaining services updated with the latest security patches.
 - **File System Monitoring:** Implement file integrity monitoring tools to detect unauthorized modifications or additions to critical system files and directories, including the user's home directory.
 - **Data Loss Prevention:** Implement data loss prevention (DLP) measures to protect sensitive data identified in the user's home directory. This

may include access controls, encryption, and data backups.

- **Security Awareness and Training:**

- **User Education:** Educate users about the importance of strong passwords, recognizing phishing attempts, and avoiding suspicious links or attachments.
- **Incident Response Planning:** Develop and regularly test an incident response plan to ensure that security incidents are handled effectively and efficiently.

By implementing these recommendations, organizations can proactively address potential vulnerabilities, strengthen their security posture, and mitigate the risks associated with cyber threats.

References

- curl command documentation from: <https://curl.se/docs/>
- GNU sed manual: <https://www.gnu.org/software/sed/manual/>
- **Network security:** (Used in Discussion to provide context for network security concepts and best practices)
 - OWASP (Open Web Application Security Project): <https://owasp.org/>
 - SANS Institute: <https://www.sans.org/>
- **Cybersecurity best practices:** (Used in Conclusion and Recommendations to support the need for a comprehensive security approach)
 - NIST Cybersecurity Framework: <https://www.nist.gov/cyberframework>
 - SANS Institute resources: <https://www.sans.org/>
- **Incident response:** (Used in Recommendations to provide guidance on incident response planning)
 - CERT (Computer Emergency Response Team): <https://www.cert.org/>